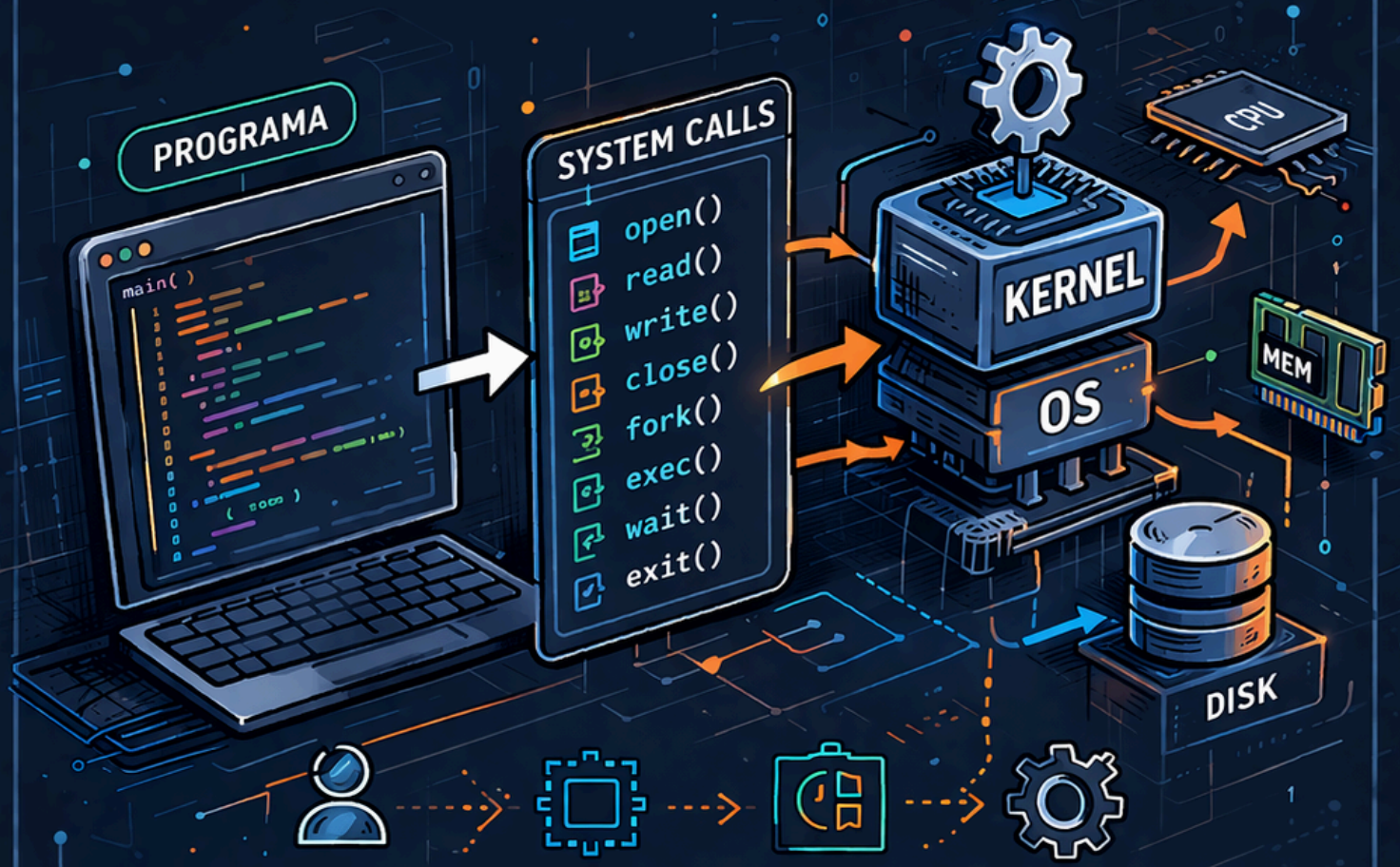
SISTEMAS OPERATIVOS

LABORATORIO 2

SISTEMA • SYSTEM CALLS • PROGRAMA



USUARIO — PROCESO — SISTEMA OPERATIVO — HARDWARE

PROFESORES:

Carlos Buckle

Francisco Páez

ESTUDIANTES:

- Agustín Gigliotti
- Martina Liberatori
- Moises Kutnich
- Santiago Gabet

AÑO: 2026

ASIGNATURA: Sistemas Operativos

1. programa saludar.c:

Modificamos el archivo para que acepte argumentos desde la terminal. Usamos argc para contar los argumentos y argv[] para leer el nombre que escribes. Si el usuario pasa un nombre, el programa imprime "hola" seguido de ese nombre. Ejecutamos bin/hola (nombre) (El programa responde: hola (nombre).

Usamos strace para ver qué le pide el programa al Sistema Operativo (Kernel). Cuando el programa llega a la línea del printf(), la computadora ejecuta internamente una system call llamada write.

<u>Funcionamiento:</u> Recibe tres datos principales:

1. El destino: El número 1 (que significa la pantalla/salida estándar).
2. El mensaje: El texto "hola [nombre]".
3. El tamaño: Cuántas letras (bytes) tiene ese mensaje.

¿Por qué tantas llamadas para un código tan simple?

Cuando ejecutas bin/hola, el sistema operativo tiene que:

1. Leer el archivo del disco.
2. Cargar librerías de lenguaje C.
3. Configurar la memoria.
4. Finalmente, ejecuta el código.

2. programa bin/fail:

Teníamos un programa ejecutable llamado fail que no funcionaba correctamente (imprimía "error" y se cerraba). No teníamos el código fuente.

Ejecutamos strace bin/fail para observar las llamadas al sistema (system calls). Al analizar el flujo de ejecución, identificamos el punto exacto de la falla:

- Llamada identificada: openat(AT_FDCWD, "archivo.txt", O_RDONLY).
- Resultado de la llamada: -1 ENOENT (No such file or directory).
- Interpretación: El programa intentaba abrir un archivo llamado archivo.txt que no existía en la carpeta. Al recibir este error, el programa estaba programado para imprimir "error" y finalizar.

Aplicamos la solución directamente desde la terminal:

1. Usamos el comando touch archivo.txt para crear un archivo vacío con ese nombre.
2. Al ejecutar bin/fail nuevamente, el programa imprimió "ok".

El programa quiere hacer lo que el sistema operativo le permite. Si una syscall falla y el programa no sabe cómo manejarlo, se produce un error de ejecución.

3. programa mycopy.c:

Permite copiar un archivo en otro utilizando dos modos de funcionamiento:

- Modo s: usa llamadas al sistema (open, read, write, close).
- Modo f: usa funciones de biblioteca (fopen, fread, fwrite, fclose).

El programa recibe tres parámetros: el modo de ejecución, el archivo origen y el archivo destino.

Se controlaron errores importantes:

- Si el archivo origen no existe, el programa finaliza mostrando el error con perror.
- Si el archivo destino ya existe, la creación falla, también mostrando el error.

La copia se realiza utilizando un buffer de tamaño fijo, leyendo y escribiendo el archivo en bloques hasta finalizar.

Luego se comparó el comportamiento de ambos modos utilizando strace -c.
Se observó que:

- En el modo s, se realizan muchas llamadas al sistema (read y write), ya que cada operación accede directamente al kernel.
- En el modo f, las funciones de biblioteca utilizan buffers internos, lo que reduce la cantidad de llamadas al sistema.